

Context Routing for AI Coding Agents

Gunther Brunner

2026-05-13

AI coding agents increasingly fail not because relevant knowledge is unavailable, but because it is poorly allocated: too much, too stale, retrieved from the wrong authority, positioned where the model attends weakly, or exposed through the wrong trust boundary. Context windows are storage budgets, not attention guarantees. A million-token window lets an agent carry more material; it does not imply that the model will retrieve, prioritize, and apply the right evidence under task pressure.

This paper surveys academic evidence on long-context degradation, retrieval-augmented code generation, repository-level retrieval, structural code search, and distractor sensitivity, alongside practitioner designs from source vendoring, agentic search, code indexing, repo maps, managed context engines, and progressive-disclosure skills. The synthesis is simple: no single context strategy dominates across model, task, repository, package, privacy, freshness, and latency conditions. The correct unit of design is a context-routing policy.

We therefore propose the context router: an explicit policy layer that selects, per task and knowledge object, among vendored source, lazy source fetch, documentation retrieval, semantic or structural search, packaged snapshots, skills, and agent-mediated inspection. We formalize the routing problem, separate the design space into orthogonal axes, identify failure modes and policy signals, define source authority and provenance requirements, outline a router-level evaluation agenda, and map current standards onto the abstraction. The paper is motivated by work on ingraft, but its primary claim is broader: context routing should become an inspectable, overrideable, standards-based layer rather than an opaque feature hidden inside individual coding agents.

Table of contents

1	Introduction	3
2	Method and Scope	4
2.1	Search and inclusion criteria	4
2.2	Evidence hierarchy	5
3	Definitions and Problem Formulation	5
3.1	Core definitions	5

3.2	Utility model	6
3.3	Policy signals	7
4	Why Context Allocation Is Hard	7
4.1	Context windows are not attention guarantees	7
4.2	Retrieval can help or hurt	8
4.3	Structure matters	10
4.4	Wrong context has concrete harms	10
4.5	Context changes during the agent loop	11
5	Design Space	12
5.1	Axis 1: placement	12
5.2	Axis 2: representation	12
5.3	Axis 3: selector	13
5.4	Axis 4: timing	13
5.5	Axis 5: dominant constraints	13
6	Practitioner Strategies as Operating Zones	15
6.1	Effect as the originating case	16
6.2	Always-on indexes versus invoked skills	16
6.3	Source is not always better than docs	17
6.4	Agentic search versus indexing is a break-even problem	18
7	Router Design	18
7.1	Inputs and outputs	18
7.2	Strategy scoring	19
7.3	Source authority and conflict resolution	19
7.4	Provenance and context lockfiles	20
7.5	Trust policy	21
7.6	Explainability requirements	22
7.7	Worked routing examples	23
8	Standards and Interoperability	23
9	Evaluation Agenda	24
9.1	Strategies to compare	24
9.2	Task families	25
9.3	Metrics, ablations, and falsification	25
10	Case Study: ingraft	26
10.1	Scope	26
10.2	From package routing to knowledge routing	26
10.3	ingraft limitations	27
11	Threats to Validity and Counterexamples	27
12	Predictions	27
13	Conclusion	28

14 Appendix A: Representative Tool Snapshot, May 2026	28
15 Appendix B: Paper Matrix	30
References	32

1 Introduction

AI coding agents are now useful enough that their failures matter. Developers ask them to patch production repositories, migrate frameworks, inspect failing tests, answer questions about large codebases, and integrate unfamiliar libraries. In these settings, the bottleneck is rarely “no relevant information exists.” The bottleneck is context allocation: which information should enter the agent’s working set, in which representation, through which trust boundary, at what point in the loop, and under what budget.

The central claim of this paper is:

No single context strategy dominates. The correct unit of design is a context-routing policy.

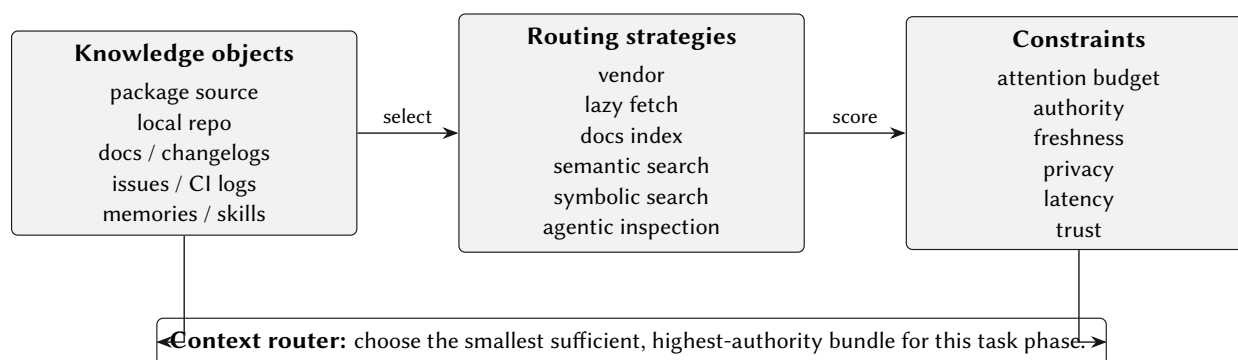


Figure 1: Context routing at a glance. The paper’s thesis is not that one context source wins; it is that coding agents need an explicit allocation policy over objects, routes, and constraints.

That claim is deliberately broader than source vendoring or any single product architecture. A coding agent needs many kinds of knowledge: package source, repository source, local conventions, issue threads, stack traces, failing test logs, API docs, examples, migration guides, previous decisions, generated summaries, dependency lockfiles, CI logs, design docs, architecture diagrams, code-search indexes, human instructions, and memory across sessions. A package is only one context object.

The more general abstraction is:

A context router selects a representation, retrieval mechanism, trust boundary, and loading schedule for each knowledge object relevant to a coding task.

In that framing, ingraft is not the proof of the thesis. It is a case study: ingraft implements the package-knowledge slice of a larger context-allocation problem. For third-party package knowledge, the router may choose vendored source, lazy source fetch, docs retrieval, semantic search, packaged snapshots, skills, or agentic inspection. For other knowledge objects, the same routing logic applies to issue trackers, project docs, CI logs, API specs, or local architecture notes.

The immediate product trigger was Maxwell Brown’s Effect post on using git subtrees for coding-agent context (Brown 2026). Brown’s short version was “Coding agents rely on patterns and examples, not descriptions” and “give your agent real library code to learn from.” That claim is intentionally narrower than this paper’s thesis. It does not settle the general problem; it exposes it: if subtree is the right route for Effect, what policy decides when subtree is right, when docs are better, and when neither should be loaded?

This paper contributes five things:

1. A problem formulation for coding-agent context allocation.
2. An evidence synthesis that distinguishes peer-reviewed research, preprints, vendor engineering posts, practitioner experience, standards, and the author’s interpretation.
3. A multidimensional design space for context strategies: placement, representation, selector, timing, and constraints.
4. A context-router design model covering policy signals, source authority, provenance, trust classes, task-phase routing, and context lockfiles.
5. A router-level evaluation agenda with ablations and falsification tests.

The intended result is not a larger landscape memo. It is a sharper argument: context engineering for coding agents is routing under attention, authority, freshness, privacy, and cost constraints. Existing tools each solve one region of that design space. A router is the abstraction that makes those partial solutions composable.

2 Method and Scope

This paper is a dated synthesis, not an exhaustive census. The review covers materials visible through May 13, 2026, with most tool and standards claims checked against public documentation, repositories, specifications, or vendor posts available during March-May 2026.

2.1 Search and inclusion criteria

Sources searched included arXiv, ACL Anthology, ACM/IEEE proceedings where available, OpenReview, GitHub, vendor documentation, engineering blogs, standards websites, tool registries, Hacker News, practitioner podcasts, and industry surveys.

Papers were included when they addressed at least one of these questions:

- long-context evaluation for language models or coding models;
- repository-level code generation, completion, repair, or localization;
- retrieval-augmented code generation;
- semantic, symbolic, graph, or AST-based code retrieval;
- agent scaffolding, context management, compression, or distractor behavior.

Tools were included when they affected how coding agents obtain repository, library, documentation, organizational, or procedural context. The appendix is a representative May 2026 snapshot, not a complete inventory. Tool counts and product features are volatile.

Excluded materials include pure model benchmarks with no context intervention, general-purpose RAG systems with no code or agent relevance, and abandoned tools unless they were historically influential.

2.2 Evidence hierarchy

The paper uses heterogeneous evidence. To keep claims calibrated, Table 1 separates the role of each evidence class.

Table 1: Evidence classes and their role in the argument.

Evidence class	Examples	How it is used
Peer-reviewed empirical papers	Lost in the Middle, RULER, RepoBench, SWE-bench, GraphCoder	Mechanisms and benchmark results with comparatively strong evidential weight
Preprints and technical reports	LongCodeBench, cAST, Context Rot, newer repo-retrieval papers	Emerging findings, cited with venue caveats
Vendor engineering posts	Anthropic, Cursor, Cognition, Augment, Sourcegraph	Evidence of deployed design patterns and product constraints, not independent proof
Practitioner essays and podcasts	Effect, Cline, aider, HumanLayer, Ptacek, Ronacher	Operating experience and design disagreement
Tool documentation and standards	MCP, Agent Skills, AGENTS.md, llms.txt, SCIP	Implementation facts and interoperability surfaces
This paper’s synthesis	router thesis, policy model, evaluation design	Interpretation that should be tested, not treated as direct empirical result

Two terms are used carefully:

- **Academic and technical works** means peer-reviewed papers, workshop papers, arXiv preprints, benchmark reports, and technical reports. It does not imply every cited work is peer-reviewed.
- **Representative tool snapshot** means a dated inventory of visible major tools. It is not complete and will age.

Practitioner evidence shows convergence around a shared problem, not consensus on the solution. The field agrees that context selection matters; it disagrees on where selection should happen.

3 Definitions and Problem Formulation

3.1 Core definitions

Context is any information made available to an agent during a task. It includes prompt tokens, files the agent can read, tool results, memory, indexes, skills, instructions, logs, and external service responses.

Context object is a knowledge unit that might be relevant: a package, file, repository, issue, stack trace, lockfile entry, API document, design note, generated summary, or skill.

Context bundle is the concrete material exposed to the agent for a task: selected files, docs, snippets, index hits, rules, provenance notes, and tool affordances.

Routing strategy is a way of exposing a context object: vendor it into the workspace, fetch it lazily, retrieve docs, query a graph index, pack a repo, activate a skill, or let an agent inspect through shell tools.

Context router is a policy $\pi(t, m, r, k) \rightarrow s$: given a task t , model or agent capability m , repository state r , and knowledge object k , select a routing strategy s .

Source authority describes which source should win for a specific question: the lockfile for installed version, official docs for public API intent, source and tests for behavior, local code for project convention, and human instructions for constraints.

Context provenance records where a bundle came from, which version it represents, when it was fetched, how trustworthy it is, and what it is intended to be used for.

3.2 Utility model

A lightweight model is enough to make the tradeoff explicit. For a coding task t , model or agent m , repository state r , knowledge object k , and candidate strategy s , the router wants high expected success and low total cost:

$$\begin{aligned}
 U(s \mid t, m, r, k) = & P(\text{success} \mid t, m, r, C_s) \\
 & - \lambda_1 \text{tokens}(C_s) - \lambda_2 \text{latency}(s) - \lambda_3 \text{privacyRisk}(s) \\
 & - \lambda_4 \text{staleness}(s) - \lambda_5 \text{distractorRisk}(C_s) - \lambda_6 \text{trustRisk}(s)
 \end{aligned}$$

The context router chooses the strategy with the highest expected utility under observable signals. The exact weights are product and organization specific, but the terms make clear why “more context” is not the objective.

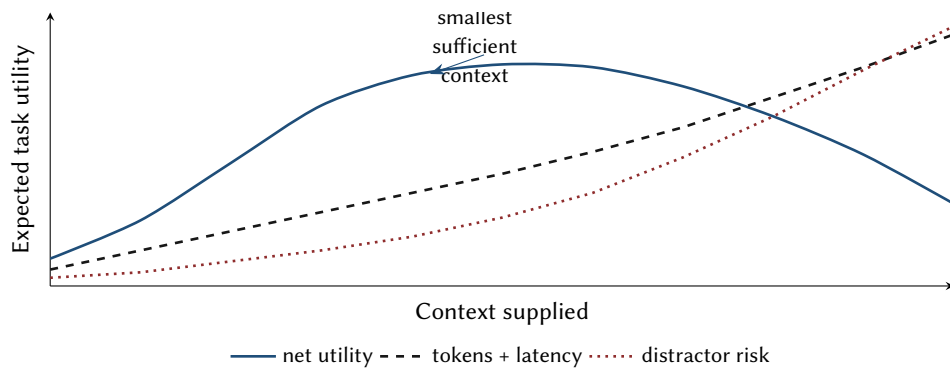


Figure 2: The router optimizes expected utility, not raw context size. More context helps until authority and relevance are outweighed by token, latency, staleness, trust, and distractor costs.

The operational principle is **smallest sufficient context**:

Context is sufficient when it lets the agent identify the relevant authority, produce a correct change, and verify it without introducing materially misleading distractors.

Smallest does not mean fewest tokens. It means lowest total context cost across tokens, latency, privacy, staleness, trust, and distractor risk. Sometimes the lowest-cost context is a small doc page. Sometimes it is a large but version-matched source tree.

3.3 Policy signals

The router cannot observe expected success directly, so it uses signals. The signals in Table 2 are enough to turn “router” from metaphor into a policy surface.

Table 2: Observable signals for a context-routing policy.

Signal	Why it matters
Package appears in lockfile	Fixes version-specific context and avoids latest-doc drift
Import frequency	Estimates package centrality and likely value of deeper source
Package source size	Affects vendoring, packing, indexing, and distractor cost
Docs availability and quality	Makes docs-first viable for public API tasks
API volatility	Increases value of fresh docs, changelogs, or source
Local modifications	Penalizes stale indexes and remote snapshots
Model context window and measured effective length	Bounds what can be reliably carried
Agent tool capability	Determines whether agentic search is viable
Privacy policy	Rules out cloud retrieval or remote indexing
Task type	Bugfix, migration, greenfield integration, test repair, security review, or refactor
Failure history	Repeated failures justify deeper context or a strategy switch
Repository size and churn	Determines index/search break-even
Language ecosystem support	Determines whether AST, LSP, or SCIP retrieval is available

4 Why Context Allocation Is Hard

4.1 Context windows are not attention guarantees

Long windows increase storage, not reliable computation over every token. N. F. Liu et al. (2024) established the now-canonical positional failure: models perform best when relevant information is near the beginning or end of the context and degrade when evidence is buried in the middle. Hsieh et al. (2024) showed that advertised context length can overstate effective context length, especially on multi-hop synthetic tasks.

LongCodeBench extends the problem to realistic code comprehension and repair (Rando et al. 2025). It evaluates LongCodeQA and LongSWE-Bench across 32K, 64K, 128K, 256K, 512K, and 1M context brackets where supported. The precise headline is:

On LongSWE-Bench, Claude 3.5 Sonnet solved 29% of issues at 32K context and 3% at 256K; it was not a 1M-token Claude result. On LongCodeQA, Qwen2.5 peaked at 70.2% at 512K and fell to 40.0% at 1M.

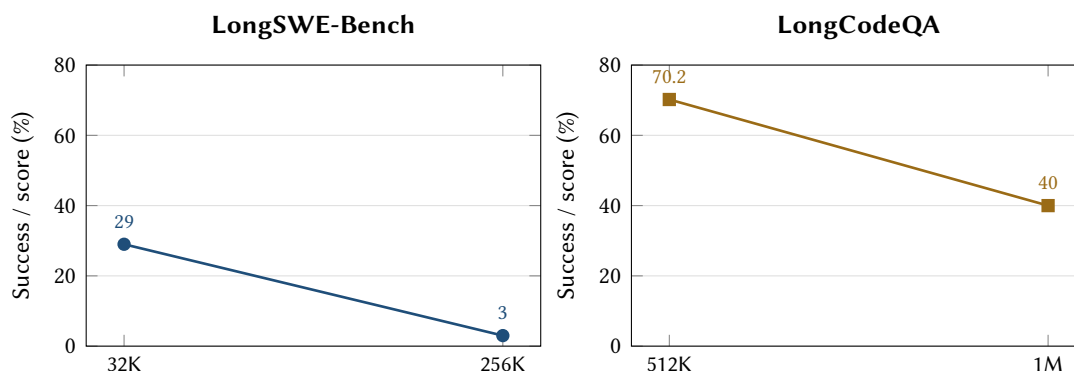


Figure 3: LongCodeBench examples used in this paper. Claude 3.5 Sonnet drops on LongSWE-Bench from 29% at 32K to 3% at 256K; Qwen2.5 drops on LongCodeQA from 70.2% at 512K to 40.0% at 1M. The point is calibration, not a universal threshold.

The newer context-rot literature sharpens the same point. Chroma’s technical report evaluates 18 models and isolates cases where increasing input length alone makes outputs less reliable, including semantic needle variants, distractors, LongMemEval comparisons, and repeated-word copying tasks (Hong, Troynikov, and Huber 2025). NoLiMa extends needle-in-a-haystack beyond literal matching and reports severe drops when questions and needles have little lexical overlap, even when short contexts are nearly solved (Modarressi et al. 2025). AbsenceBench adds a different failure class: models that can retrieve present evidence may still fail to identify missing information, including omitted lines in GitHub pull-request diffs (Fu et al. 2025).

The defensible conclusion is not “1M always fails.” The conclusion is that advertised context length is an upper bound, not an operating target. Routing budgets should be calibrated per model, task class, and representation, ideally by local evals or conservative defaults.

4.2 Retrieval can help or hurt

Z. Z. Wang et al. (2025) asks directly whether retrieval can augment code generation. Its answer is conditional: high-quality retrieved contexts improve some settings, especially weaker models and basic programming tasks, while retrievers still struggle when lexical overlap is limited and generators do not always integrate extra context effectively. Asai et al. (2024) reaches a similar architectural lesson from a broader RAG setting: indiscriminate fixed retrieval can reduce versatility and produce unhelpful generations; retrieval should be adaptive.

The router should not ask “RAG or not?” It should ask which retrieval substrate has the lowest expected miss, distractor, staleness, and latency cost for this task.

That means semantic and non-semantic search should be treated as different instruments, not rival slogans. BM25 remains a strong lexical baseline because exact terms, identifiers, and error strings often are the task (Robertson and Zaragoza 2009). Learned sparse retrieval such as SPLADE keeps lexical auditability while adding expansion terms (Formal, Piwowarski, and Clinchant 2021). Dense retrieval and text embeddings are useful when intent does not share surface form with the target, but benchmarks such as BEIR and MTEB

show that retrieval quality is highly task- and distribution-dependent (Thakur et al. 2021; Muennighoff et al. 2023). CodeSearchNet made the natural-language-to-code gap explicit for semantic code search (Husain et al. 2019), while GraphCodeBERT shows why code structure and data flow matter beyond flat token embeddings (Guo et al. 2021). Late-interaction retrievers such as ColBERT occupy a useful middle ground: they retain token-level matching signals while allowing document representations to be precomputed (Khattab and Zaharia 2020). ColBERTv2 and PLAID make the deployment point stronger by reducing late-interaction storage and serving cost without giving up the core matching advantage (Santhanam et al. 2022a; Santhanam et al. 2022b). Mixedbread’s mxbai work is relevant as practitioner evidence for both dense embeddings and ColBERT-style reranking, and mgrep is a concrete coding-agent product claim that semantic search can reduce token usage relative to repeated grep loops while still preserving grep as the exact match fallback (Lee et al. 2024a; Lee et al. 2024b; Mixedbread 2026). Multi-function embedding systems such as BGE-M3 push in the same direction by supporting dense, sparse, and multi-vector retrieval modes inside one model family (Chen et al. 2024).

Recent code and technical-document retrieval benchmarks make the same lesson more concrete for software agents. FreshStack argues for benchmarks built from recent technical documentation, where off-the-shelf retrievers can lag far behind oracle context on niche stacks (Thakur et al. 2025). COIR separates code retrieval into multiple task families rather than treating “code search” as one problem (Li et al. 2024). CoSIL frames issue localization as graph-guided repository search rather than flat chunk retrieval (Jiang et al. 2025). For routing, these papers matter because they move the selector question from “semantic search or grep?” to “which retrieval substrate matches this task, authority, and failure mode?”

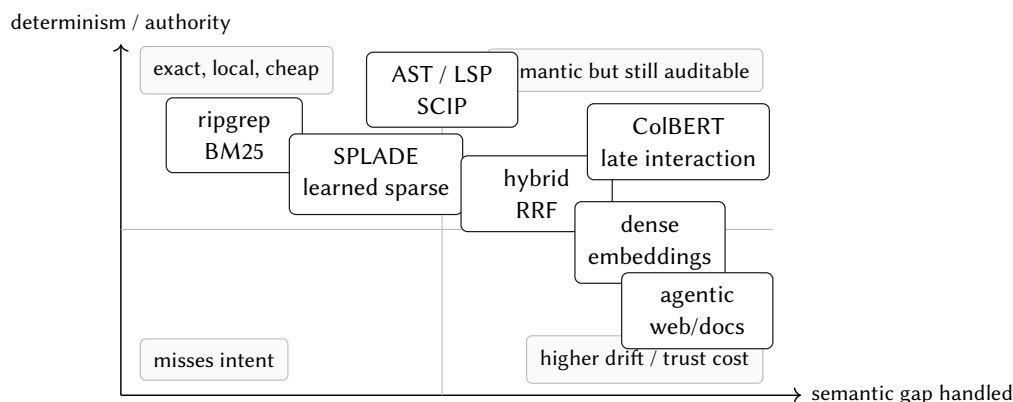


Figure 4: Semantic and non-semantic search are complements. Exact lexical search is cheap and deterministic; dense and late-interaction search bridge intent gaps; graph and symbol search preserve code authority; hybrid retrieval is often the pragmatic route.

Table 3: Retrieval substrates and their common failure modes.

Retrieval type	Typical failure mode
Lexical grep	Misses semantic matches and renamed concepts
Semantic vector search	Retrieves plausible but wrong chunks
Graph retrieval	Depends on graph quality and language support
AST chunking	Can miss runtime behavior and dynamic dispatch
Docs retrieval	Docs may be stale, incomplete, or for the wrong version

Retrieval type	Typical failure mode
Agentic search	Slow, non-deterministic, and tool-call heavy
Hybrid search	More accurate but harder to debug and reproduce

4.3 Structure matters

Repository-level work consistently shows that flat text is a weak abstraction for code. F. Zhang et al. (2023), Shrivastava et al. (2023), Ding et al. (2024), Ding et al. (2023), Liang et al. (2024), Phan et al. (2025), and W. Liu et al. (2024) all point toward the same lesson: imports, definitions, call relationships, type information, symbols, and file structure provide routing signals that raw chunks do not. Y. (Sherry). Zhang et al. (2025) adds a newer preprint result for AST-aware chunking.

The careful implication is not “AST plus graph must always be the default.” It is: for repository-scale retrieval, the prior should favor structural retrieval whenever the language ecosystem and latency budget permit it.

4.4 Wrong context has concrete harms

Context quality fails through recognizable mechanisms. Table 4 maps each failure mode to a router response.

Table 4: Failure modes that motivate routing.

Failure mode	Description	Router response
Attention dilution	Relevant tokens are present but weakly attended	Reduce bundle size; place high-value context early or late
Distractor interference	Plausible irrelevant context changes the reasoning path	Prune aggressively; prefer version-specific authority
Representation mismatch	Docs explain concepts but not usage patterns	Fetch examples, source, or tests
Version skew	Retrieved docs/source do not match installed package	Resolve through lockfile and provenance
Retrieval miss	Search fails to fetch the necessary file or symbol	Expand via graph, AST, LSP, or agentic search
Context staleness	Index reflects old repo state	Prefer live grep or incremental index
Trust contamination	External docs contain malicious or irrelevant instructions	Quote, sanitize, label provenance, sandbox tools
Cost blowup	Agent spends many turns searching	Precompute repo maps or local indexes

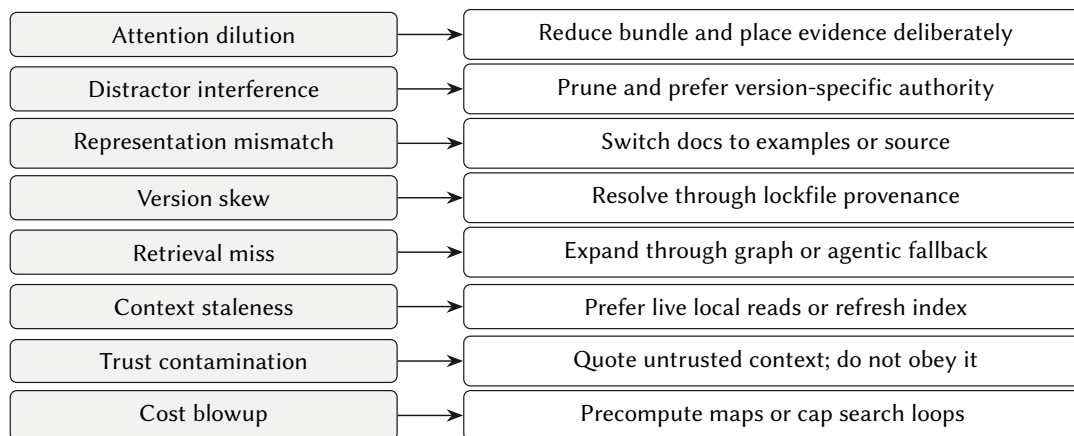


Figure 5: Wrong context has concrete failure modes. A router is useful only if each failure maps to an intervention: prune, switch representation, refresh, change authority, or stop searching.

Examples are easy to find in everyday agent work: the agent uses latest docs for an older installed package, imports a private symbol after reading source, acts on a StackOverflow answer for the wrong major version, searches a stale index while local uncommitted changes matter, follows instructions embedded in third-party docs, or receives a whole repo pack and overlooks the relevant file in the middle. These are not model failures alone. They are allocation failures.

4.5 Context changes during the agent loop

Context routing is online, not a one-time preprocessing step. Each phase of an agent loop wants different material.

Table 5: Task-phase routing.

Phase	Useful context
Orient	AGENTS.md, repo map, architecture summary, task brief
Locate	Code search, symbols, imports, stack traces, issue references
Plan	Relevant local files, docs, examples, package conventions
Edit	Exact files, types, tests, local wrappers, public API contracts
Verify	Test logs, CI output, type errors, runtime traces
Repair	Failing trace, package source, issue history, migration docs
Summarize	Decision log, changed files, unresolved risks, provenance

Phase	Rules	Repo map	Search	Files	Package src	Tests/logs
Orient	3	3	1	1	0	0
Locate	1	2	3	2	1	1
Plan	2	1	2	3	2	1
Edit	1	0	1	3	2	1
Verify	1	0	1	2	1	3
Repair	1	1	3	3	3	3
Summarize	2	2	0	2	0	1

Figure 6: Task-phase context heatmap. Darker cells indicate higher expected usefulness. Context routing is an online policy: the right bundle changes as the agent orients, locates, edits, verifies, repairs, and summarizes.

This aligns with Anthropic’s context-engineering framing: curation and maintenance of useful tokens happens throughout inference, not only at prompt construction time (Anthropic 2025a). Emerging OSS-focused context engineering work makes the same point at repository scale: agent configuration files, guidance documents, and tool manifests vary widely, which means context policy itself becomes part of the software architecture rather than an afterthought (Mohsenimofidi et al. 2025).

5 Design Space

The earlier “seven primitives” vocabulary is useful as shorthand, but it mixes dimensions. Vendoring, indexing, retrieving, packing, skills, annotations, and agentic reads are common bundles across several independent axes. A more useful taxonomy separates the axes first.

5.1 Axis 1: placement

Table 6: Where context lives.

Placement	Examples
In prompt	Pasted file, repo pack, selected snippets
On disk in workspace	Vendored source, cloned dependency, exported docs
Local cache	OpenSrc-style package cache, tool-managed snapshots
Local index	Sourcebot, Serena, local vector DB, SCIP/LSP graph
Remote service	Context7, Augment, Glean, hosted semantic search
Model or tool memory	Skills, summaries, persistent notes, agent memory

5.2 Axis 2: representation

Table 7: How context is represented.

Representation	Examples
Raw source	Vendored repository, fetched package source
Human docs	API docs, README, migration guides, changelogs
Symbol graph	SCIP, LSP, AST, call/reference graph
Embedding chunks	Vector search over files, docs, or issues
Summaries	Repo maps, architecture notes, skills, memory
Executable tools	MCP tools, scripts, local CLI commands
Instructions	AGENTS.md, CLAUDE.md, project rules

5.3 Axis 3: selector

Table 8: Who or what selects context.

Selector	Examples
Human	@file, explicit package, manual doc link
Static policy	allowlist routes effect to vendor
Heuristic router	import count, package size, privacy class
Learned retriever	embedding search, reranker
Graph retriever	symbol, dependency, call/reference expansion
Agentic search	grep/read/bash loop
Oracle/evaluator	benchmark-only upper bound

5.4 Axis 4: timing

Table 9: When context is loaded.

Timing	Examples
Always loaded	AGENTS.md, compact repo guidance
Session start	Repo map, recent decision log
Task start	Package docs, relevant tests, task issue
Just in time	Agent calls search, router fetches source
After failure	Fetch package source after stack trace points there
Compacted later	Summarize stale context for continuation

5.5 Axis 5: dominant constraints

Table 10: Constraints that dominate route choice.

Constraint	Examples
Token budget	Long-context degradation, prompt cost

Constraint	Examples
Latency	Agentic search and remote retrieval delays
Privacy	Local-only code, no cloud index, no remote docs
Freshness	Local changes, docs drift, moving issues
Trust	MCP/tool/skill supply chain, prompt injection
Determinism	Reproducible context versus live retrieval
Version specificity	Lockfile-resolved package versions

The old primitives can now be expressed more precisely:

- **Vendoring** = workspace placement + raw source representation + static or heuristic selection + task-time loading + strong reproducibility + local privacy.
- **Indexing** = local or remote placement + symbol/embedding representation
 - learned or graph selector + low-latency query + staleness management.
- **Skills** = memory placement + instruction/tool representation + metadata-based selector + progressive disclosure + high token efficiency.
- **Repo pack** = prompt placement + raw or summarized representation + human or static selector + one-shot timing + weak freshness after creation.

For implementation, these axes become a route descriptor rather than a prose taxonomy.

Table 11: Route-descriptor axes for a context object.

Axis	Question	Example values
Object kind	What is being routed?	Package source, repo subtree, doc page, API spec, ticket, skill, live endpoint
Materialization	Where does it live for the task?	Committed, cached, packed, mounted, remote-only
Representation	In what form is it exposed?	Raw source, structural summary, prose doc, metadata envelope, search hit
Access path	How does the agent reach it?	Direct read, local grep, local index, remote retrieval API, MCP tool
Freshness mode	How stable is it?	Pinned snapshot, refreshable cache, live mutable
Authority/trust	How much should the agent rely on it?	Canonical, upstream authoritative, derived, advisory, unverified
Privacy scope	What boundary constrains it?	Repo-safe, local-only, restricted, remote-call-required

6 Practitioner Strategies as Operating Zones

The practitioner landscape is not a consensus. It is structured disagreement. Each camp has a correct operating zone and a failure mode.

Table 12: Practitioner camps, assumptions, failure modes, and routing implications.

Camp	Claim	Evidence	Hidden assumption	Failure mode	Router implication
Source vendoring	Agents learn patterns better from source than prose	Effect subtree argument (Brown 2026)	Source is available, navigable, version-matched, and not huge	Internal APIs, large repos, poor comments, private access	Vendor high-centrality packages with manageable source and high idiom value
Agentic search	Let the model inspect live files instead of maintaining an index	Claude Code and Cline practice (S. (swyx). Wang and Trotta 2025; Baumann 2025)	Agent has strong tools and time budget	Latency, non-determinism, repeated search loops	Prefer live search when local files exist and freshness matters
Pre-indexing	Large repos need fast semantic or structural context	Cursor and Sourcegraph/Amp claims (Cursor 2026; Sourcegraph 2025)	Index is fresh, private enough, and amortized over many tasks	Staleness, privacy, setup cost	Index when repo size and query frequency justify maintenance
Repo maps	Summaries preserve structure under tight budgets	aider tree-sitter repo map (Gauthier 2023)	Symbols/signatures are enough for orientation	Missing details and runtime behavior	Use maps for orientation, then expand to exact files
Skills and rules	Procedural knowledge should be progressively disclosed	Agent Skills and CLAUDE.md practice (Anthropic 2025b; HumanLayer (Kyle) 2025)	Knowledge is stable and reusable	Overbroad instructions and hidden script risk	Use skills for procedures, conventions, and reusable workflows

Camp	Claim	Evidence	Hidden assumption	Failure mode	Router implication
Always-on docs indexes	A compact project instruction file can reliably point agents at versioned local docs	Vercel Next.js agent evals (Gao 2026a)	The index is small, accurate, and points to local docs the agent can read	Context bloat; stale copied docs; agent ignores project instruction files	Put high-value retrieval pointers in AGENTS.md, and reserve skills for explicit workflows
Managed enterprise context	Org search needs ACLs, multi-repo scale, and governance	Glean, Augment, Sourcegraph, DeepWiki-style systems	Cloud or managed index is allowed	IP leakage, opaque ranking, vendor lock-in	Route through managed systems only when policy permits

6.1 Effect as the originating case

Effect is the motivating case for ingraft and for this paper. Brown’s post makes a narrow but important source-vendoring claim: for an idiom-heavy TypeScript framework, human-facing docs can be too lossy for coding agents, while source, tests, examples, and module structure preserve the patterns agents need (Brown 2026). The recommended route is git subtree under a read-only reference directory, with agent instructions and editor/tool ignores so humans are not overwhelmed by vendored code.

The public article should not be treated as a controlled benchmark report. It does not publish task definitions, model list, run counts, or pass/fail tables. Its evidential role here is practitioner operating experience: for Effect, the authors report that locally available source is more useful to agents than fragmented web fetches, human docs, or compiled `node_modules` surfaces. That is enough to justify source vendoring as a candidate route, not enough to make it the global default.

This is exactly the router opening. If source vendoring wins for Effect, the hidden variables are high package centrality, idiom density, navigable TypeScript source, useful tests and examples, manageable repository size, and need for version-specific source. The router implication is not “always vendor”; it is “vendor when source authority and idiom value outweigh repository impact and distractor risk.”

6.2 Always-on indexes versus invoked skills

Vercel’s Next.js team published a useful negative result for the router thesis: on a hardened Next.js 16 eval suite, an on-demand docs skill was often not invoked and did not improve over the no-docs baseline, while explicit instructions improved results but were wording-sensitive (Gao 2026a). Their best condition was a compressed AGENTS.md docs index: the project file kept a small map of version-matched docs always visible, and the agent read the actual docs from a local `.next-docs/` directory when needed. The reported pass rates were 53% for baseline, 53% for default skill use, 79% for skill plus explicit instruction, and 100% for the AGENTS.md docs index.

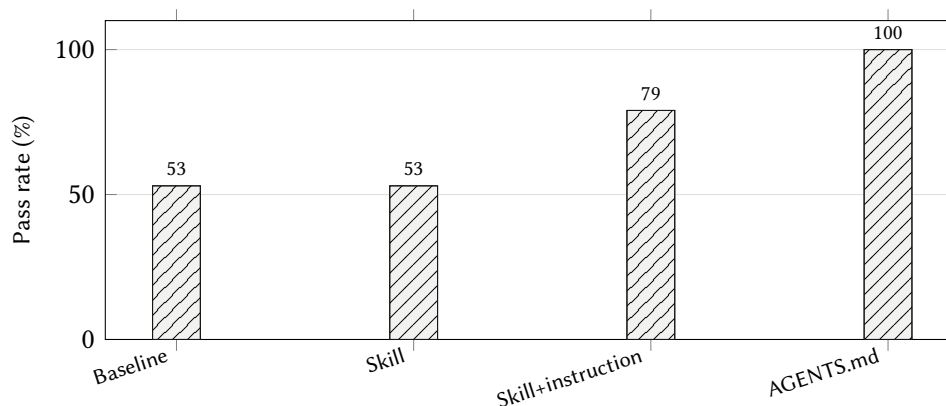


Figure 7: Vercel’s reported Next.js agent eval results. The lesson for routing is selector and timing: an always-on compact docs index outperformed an on-demand skill in this suite.

This is vendor evidence, not an independent controlled study, and it is specific to Next.js tasks and the harnesses tested. Still, it sharpens the routing model: there are cases where the best route is not to load full docs into context and not to rely on skill discovery, but to expose a small always-on retrieval index that removes the agent’s decision to ask for help. Vercel’s public `next-evals-oss` repository is also a practical evaluation pattern: self-contained project fixtures with `PROMPT.md`, withheld `EVAL.ts`, paired experiments with and without `AGENTS.md`, and exported docs-impact deltas (Vercel 2026). The merged `@next/codemod agents-md` command shows the implementation route: detect the framework version, fetch matching docs, generate a compressed index, inject it between markers in `AGENTS.md` or `CLAUDE.md`, and ignore the local docs directory (Gao 2026b).

6.3 Source is not always better than docs

The Effect case is one operating zone, not a universal source-over-docs rule. The source-vendoring argument is strongest when agents need idioms, examples, type-level patterns, framework internals, and implementation behavior. Docs are often the authority for public API intent, migration guidance, deprecation warnings, and security constraints.

Table 13: Source-versus-docs decision rule.

Task need	Prefer
Learn public API quickly	Docs and curated examples
Understand behavior edge case	Source and tests
Follow idioms	Source examples plus local usage
Avoid private/internal APIs	Official docs and public type surface
Debug package internals	Version-matched source
Migrate across versions	Changelog and migration guide, then local usage
Verify generated code	Tests plus source
Understand package philosophy	Docs and design notes

6.4 Agentic search versus indexing is a break-even problem

The unresolved question is not whether indexing or agentic search is “right.” It is when the cost of maintaining an index exceeds the cost of repeated live search.

Agentic search wins when:

$$C_{\text{toolCalls}} + C_{\text{tokens}} + C_{\text{latency}} < C_{\text{indexBuild}} + C_{\text{indexDrift}} + C_{\text{privacy}} + C_{\text{retrievalErrors}}$$

Indexing wins when repository size, query frequency, and cross-file complexity are large enough to amortize index maintenance. This depends on churn, local change frequency, language tooling, privacy rules, and whether cross-repo search matters. The router should make that break-even explicit rather than choosing a side globally.

7 Router Design

7.1 Inputs and outputs

A context router takes:

- task statement and task type;
- repository state, including local changes and lockfiles;
- knowledge object metadata;
- model and tool capabilities;
- privacy and trust policy;
- history of failures or user overrides.

It outputs:

- selected route and alternatives considered;
- context bundle or tool affordance;
- provenance record;
- trust class and intended use;
- explanation and override path;
- optional context-lock update.

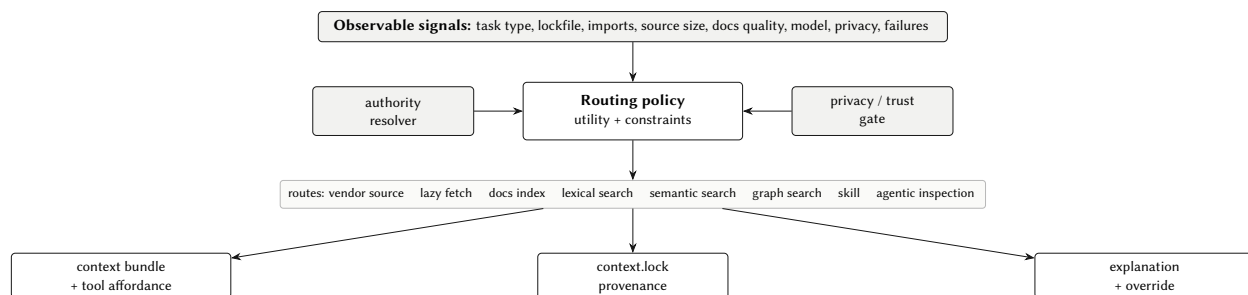


Figure 8: Context router architecture redrawn as a PDF-native diagram. The router does not merely retrieve content; it resolves authority, enforces trust policy, emits provenance, and explains the route.

7.2 Strategy scoring

The router can begin as a rule system, but the policy should expose scoring features so users can understand why a route was chosen.

Table 14: Scoring features across route families.

Feature	Vendoring	Lazy fetch	Docs	Index	Skill
Version fidelity	High if lockfile-resolved	High if resolved	Medium to high	Varies	Low unless versioned
Token efficiency	Medium	Medium	High	High	High
Agent navigability	High	High	Medium	Medium/high	Low/medium
Setup cost	Medium/high	Low	Low	High	Low
Privacy	High	High	Varies	Varies	High
Freshness	Pinned	Pinned	Fresh but may mismatch	Drift risk	Stable
Distractor risk	High if huge	Medium	Medium	Medium	Low
Best for	Deep idioms	Occasional inspection	Public API/migration	Large repos	Conventions/procedures

Default policy rules follow from the table:

- Vendor when package centrality is high, source size is manageable, privacy matters, and idioms matter.
- Lazy-fetch when a package is used occasionally but source-level inspection may be needed.
- Docs-first when the task targets public API, migration, conceptual usage, security guidance, or newly released features.
- Index when repository or package scale exceeds grep-friendly size, or cross-file relationships dominate.
- Skill when knowledge is procedural, stable, reusable, and safe to package.
- Agentic-search when local source is available and the model/tooling can inspect selectively.
- Pack when the agent lacks filesystem/search tools or the task is one-shot and offline.

7.3 Source authority and conflict resolution

A router should ask not only “where can I get information?” but “which source is authoritative for this question?”

Table 15: Source authority by question type.

Question	Best authority
What version is installed?	Lockfile and package manager metadata
What public API is intended?	Official docs and public type surface
What behavior occurs?	Version-matched source and tests

Question	Best authority
What pattern does this repo prefer?	Local codebase and AGENTS.md
What should the agent never do?	AGENTS.md, security policy, human instruction
What broke?	Failing test, log, trace, CI output
What changed between versions?	Changelog, release notes, migration guide
What symbols reference this?	LSP, SCIP, source index, grep

Conflict resolution should preserve provenance. A useful agent-facing message is not “use zod docs”; it is “this recommendation is based on local package source resolved from `bun.lock`, not latest web docs.” That distinction prevents version skew and makes human override possible.

7.4 Provenance and context lockfiles

The router should emit context plus provenance. Example bundle metadata for source:

```
context_bundle:
  source: npm
  package: effect
  version: 3.14.2
  route: vendor
  resolved_from: bun.lock
  fetched_at: 2026-05-13
  trust_class: local-readonly-source
  mutable: false
  intended_use: reference-only
```

For docs:

```
context_bundle:
  source: context7
  package: next
  version_hint: "15.x"
  route: docs
  fetched_at: 2026-05-13
  trust_class: remote-docs
  warning: verify against lockfile and local source before editing
```

A `context.lock` file is the reproducibility bridge between research and implementation. It records the strategy, source URL, package version, commit, docs version, index version, skill version, trust class, generated summaries, timestamps, and override reasons.

```
[packages.effect]
strategy = "vendor"
version = "3.14.2"
source = "github:Effect-TS/effect"
```

```
commit = "abc123"
reason = "core-framework allowlist; imported in 42 files"
trust = "local-readonly"

[packages.zod]
strategy = "docs-first"
version = "4.1.0"
docs = "context7:zod@4.1"
fallback = "fetch-source"
reason = "public API task; source size low; docs available"
trust = "remote-docs"

[tools.mgrep]
enabled = false
reason = "privacy=strict-local"
```

Table 16: Core fields for a context lockfile.

Lockfile field	Purpose
object_id	Stable identity for the routed artifact
kind	Context-object class
source_uri	Upstream origin
revision	Commit, version, timestamp, or content hash
authority	Canonical, upstream, derived, advisory, or unverified
materialization	Committed, cache, mount, live, or pack
representation	Raw, summary, metadata, index, or executable tool
privacy_class	Repo-safe, local-only, restricted, or remote
integrity	Hash, signature, or provenance attestation
phase_policy	Preferred routes by task phase
refresh_policy	Pinned, manual refresh, TTL, or live
transform_chain	Export, normalization, and summarization steps

Provenance supports version fidelity, audit logs, privacy review, benchmark reproducibility, and human overrides.

7.5 Trust policy

Security is central to routing, not an appendix. Every route has a different attack surface.

Table 17: Route-specific security risks.

Route	Risk
Docs retrieval	Prompt injection, outdated examples, wrong version
MCP server	Tool overpermission, data exfiltration, remote execution

Route	Risk
Skills	Executable scripts, hidden instructions, supply-chain attacks
Vendored source	Malicious comments, enormous distractor surface
Repo pack	Secret exposure and uncontrolled redistribution
Cloud index	IP leakage and opaque retention
Agentic search	Command misuse and accidental broad reads
Generated summaries	Untraceable hallucinated compression

A practical router should support policy modes such as:

- `strict-local`: no remote retrieval or cloud index;
- `local-executable-denied`: local files allowed, bundled scripts blocked;
- `remote-docs-allowed`: remote docs allowed but quoted as untrusted;
- `remote-code-denied`: no remote source fetch;
- `cloud-index-allowed`: managed indexes permitted;
- `scripts-require-confirmation`: skills and MCP tools cannot execute without confirmation;
- `untrusted-context-quoted`: external context is evidence, not instruction.

The ethical case for an open router follows from these requirements: transparent routing decisions, user override, local-first defaults, provenance logs, auditable lockfiles, no silent cloud upload, standards-based portability, and avoidance of vendor lock-in.

7.6 Explainability requirements

A useful routing explanation includes:

- selected strategy;
- alternatives considered;
- reason selected;
- risks;
- override command or config;
- provenance;
- expected cost.

Example:

Routed `effect` to `vendor` because it is imported in 47 files, appears in the core-framework allowlist, source size is below 25 MB, and privacy policy is `strict-local`. Alternatives: `docs-first` rejected because the task requires idiomatic patterns; `remote index` rejected by privacy policy. Override with `ingraft route effect --strategy fetch`.

Explanations are not decoration. They are how users correct the policy.

7.7 Worked routing examples

Daily framework dependency. A project uses Effect in 80 files. The task is to implement a new service using local idioms. The router vendors Effect source read-only, emits AGENTS.md guidance, and tells the agent to inspect local examples before docs.

Rare utility package. The project uses nanoid in one file and the task is unrelated. The router does nothing. If the agent asks, it fetches source lazily.

API migration. A project upgrades Next.js from 14 to 15. The router prioritizes version-specific migration docs and changelog, then local usage search, then source fallback.

Large monorepo. The task touches one service in a 500K-file monorepo. The router avoids full packing, queries structural code search or SCIP/LSP, and passes only relevant files.

Privacy-restricted enterprise repo. Cloud retrieval is disabled. The router uses local source, local docs, local symbolic search, and blocks remote MCPs.

8 Standards and Interoperability

Standards matter because routing should not be trapped inside one agent vendor. The standards landscape is increasingly centered around AAIF projects such as MCP, goose, and AGENTS.md, alongside adjacent standards and conventions such as Agent Skills, ACP, A2A, llms.txt, and SCIP. It would be wrong to imply that all of these have one governance story.

The Linux Foundation announced the Agentic AI Foundation on December 9, 2025, with founding project contributions including Anthropic’s Model Context Protocol, Block’s goose, and OpenAI’s AGENTS.md (Linux Foundation 2025). The same announcement describes MCP as a protocol for connecting models to tools, data, and applications, and AGENTS.md as a project-guidance standard for coding agents. Agent Skills has its own specification and progressive-disclosure model (Agent Skills Working Group 2025; Anthropic 2025b). ACP, A2A, llms.txt, and SCIP have separate origins and adoption paths (Zed Industries 2025; Google and Linux Foundation 2025; Howard 2024; Sourcegraph 2024).

Table 18: Standards and their routing role.

Standard or convention	Router role
AGENTS.md	Declares project-level routing policy and agent instructions
MCP	Exposes router tools to multiple agents
Agent Skills	Packages strategy-specific procedures with progressive disclosure
ACP	Lets editors invoke agents that call the router
A2A	Lets agents coordinate or delegate context work
SCIP/LSP	Supplies graph and symbol context
llms.txt	Provides docs-discovery hints, low authority unless verified

The portable stack is therefore not “one standard to rule them all.” It is: AGENTS.md for durable project policy, MCP for tool access, Agent Skills for procedural knowledge, and language/index standards for structural retrieval.

9 Evaluation Agenda

The router thesis needs a router eval. The key research question is:

Does a task-aware context router outperform fixed context strategies under equal model, budget, and tool conditions?

9.1 Strategies to compare

Table 19: Fixed strategies and router condition.

Strategy	Description
No extra context	Baseline agent with repo only
Docs-only	Version-specific docs retrieval where available
Always-on docs index	AGENTS.md/CLAUDE.md points to local versioned docs
Source vendoring	Dependency source in workspace
Lazy source fetch	Package source fetched on demand
Repo pack	Repomix-style snapshot
Semantic index	Embedding or code-search retrieval
Structural index	AST, LSP, SCIP, call/reference graph retrieval
Agentic search	Bash/grep/read loop
Skills/rules	Procedural guidance only
Router	Chooses among the above

Strategy	Unknown lib	Framework idiom	Migration	Stack trace	Multi-file	Security	Perf/debug	Test repair
No extra context	–	–	–	–	o	o	–	o
Docs-only	+	o	+	–	–	+	–	o
Always-on docs index	+	+	+	o	o	+	–	o
Source vendoring	o	+	o	+	o	o	+	+
Lazy source fetch	o	o	o	+	o	o	+	+
Repo pack	o	o	–	o	+	–	o	o
Semantic index	+	o	o	o	+	o	o	o
Structural index	o	+	o	+	+	o	+	+
Agentic search	o	+	o	+	+	o	+	+
Router	+	+	+	+	+	+	+	+

Figure 9: Evaluation lattice. Symbols encode expected route relevance: ‘+’ core condition, ‘o’ optional condition, ‘–’ weak or negative-control condition. A router benchmark should vary strategy while holding agent and task fixed.

9.2 Task families

Table 20: Task families for router evaluation.

Task family	Example
Unknown library use	Integrate unfamiliar package
Framework idiom task	Add route, middleware, schema, service, or hook
API migration	Upgrade package version
Bugfix with stack trace	Failure crosses dependency boundary
Multi-file feature	Edit across local repo
Security-sensitive change	Avoid unsafe API or pattern
Performance/debugging task	Inspect implementation behavior
Test repair	Fix tests requiring project conventions

Stratification variables should include package popularity, source size, docs quality, framework versus utility package, typed versus untyped ecosystem, repo size, public API versus internals, package version freshness, model class, and privacy setting.

9.3 Metrics, ablations, and falsification

Table 21: Router evaluation metrics.

Metric	Why it matters
pass@1 / pass@k	Task success
Tests passed	Functional correctness
Human review score	Maintainability and style
Tokens used	Context efficiency
Wall-clock latency	Practical usability
Tool calls	Loop complexity
Retrieval precision	Context quality
Provenance coverage	Auditability
Version match rate	Source/doc correctness
Regression rate	Safety
Cost per solved task	Operational relevance

Ablations should remove model awareness, privacy constraints, vendoring, docs, graph/AST retrieval, stale-context detection, and failure-triggered strategy switching. Stress tests should include stale docs, distractor context, wrong package versions, and small versus frontier models.

The thesis should be considered weakened if:

- one fixed strategy beats the router across most task/model/repo settings;
- router explanations do not help users override bad choices;
- routing overhead cancels out quality gains;
- context strategy matters less than model choice once modern agents are used;

- simpler agentic search dominates retrieval, indexing, and vendoring under realistic budgets.

Strong papers say what would falsify them. These are the falsification tests.

10 Case Study: ingraft

This paper is motivated by ingraft, an open-source CLI for package and repository context routing (Brunner 2026). The case study should be read as an implementation path, not as empirical proof that the router wins.

10.1 Scope

ingraft’s natural first slice is package knowledge:

- inspect package manager state and lockfiles;
- route core dependencies to read-only vendored source when justified;
- route long-tail dependencies to lazy fetch;
- route public API or migration tasks to docs-first;
- expose provenance and route explanations;
- write AGENTS.md guidance so agents treat vendored source as reference material and do not import from it.

This maps to the project rule already used in this repository: vendored repositories under `vendor/` are read-only reference material, not application code.

10.2 From package routing to knowledge routing

The same design generalizes to non-code knowledge, but with sharper privacy and authority rules. A Linear ticket, Notion decision page, OpenAPI spec, CI log, or Figma design note can be treated as a context object. The router can choose to vendor it as committed text, fetch it into a local cache, mount it read-only, query it through an MCP server, or summarize it into a skill.

The important design point is not “everything belongs in `vendor/`.” It is that every artifact needs an explicit route, provenance record, privacy class, and authority label.

Example workspace layout:

```
vendor/                                # committed, read-only reference material
  github/effect-ts/effect/             # source snapshot
  notion/team-handbook/index.md        # exported page
  linear/DEV-42/index.md               # ticket snapshot
  openapi/payments-api.json            # pinned API spec

.ingraft/
  config.toml                          # committed routing policy
  context.lock                         # committed or reviewed provenance lock
```

```
cache/          # gitignored lazy fetches
mounts/         # gitignored live mounts
```

10.3 ingraft limitations

The current evidence supports ingraft as a promising implementation of a repo-resident router, not as an established empirical winner. The next step is the router eval in Section 9. ingraft should also treat security as a first class feature: signed connector metadata, local-first defaults, privacy-class checks, provenance logs, and explicit user confirmation before executable skills or connector scripts run.

11 Threats to Validity and Counterexamples

Benchmark validity. SWE-bench, LongCodeBench, CodeRAG-Bench, RepoEval, CrossCodeEval, and vendor evals measure different task distributions. Improvements on one may not transfer to another.

Model drift. Results from Claude 3.5, GPT-4o, Qwen2.5, Gemini 1.5, and other models may not extrapolate to newer systems. A router should use model-aware calibration, not hardcoded thresholds.

Tooling confounds. Agent performance reflects model, scaffold, tools, prompts, editing loop, test harness, retry policy, and human oversight. It is hard to isolate context strategy alone.

Vendor telemetry. Cursor, Augment, Cognition, Greptile, and similar numbers may be directionally useful but should not be treated as independent evidence unless methodology is public.

Publication bias. Practitioner essays report successful workflows more often than failed context strategies.

Temporal instability. Tool features, standards adoption, pricing, security posture, and model context behavior change quickly. Dated snapshots should be clearly labeled.

Routing may not help much in very small repos where the whole context fits, tasks dominated by model reasoning rather than repo knowledge, highly standardized frameworks where docs are enough, environments where the agent already has excellent local tool use, security-constrained settings where remote retrieval is forbidden and vendoring is impossible, product-judgment tasks, repos with poor tests, languages with weak structural tooling, generated or minified codebases, and monorepos whose package boundaries do not match architectural boundaries.

12 Predictions

The router thesis makes testable predictions:

1. Hybrid systems will outperform single-context systems under diverse task distributions.
2. The optimal route will vary by package centrality, not merely repo size.
3. Source vendoring will help most on idiom-heavy frameworks and least on stable utility libraries.
4. Docs-first retrieval will outperform source-first on migration tasks and public API usage.
5. Graph/AST retrieval will show larger gains as cross-file dependency depth increases.
6. Aggressive retrieval will hurt more often on distractor-heavy tasks than on tightly scoped tasks.

7. User-visible routing explanations will reduce repeated failure loops by making overrides possible.

13 Conclusion

The 2026 context-engineering landscape is not a contest between “vendor source” and “use RAG.” It is a routing problem. Context windows remain attention scarce, retrieval is conditional, structure matters, authority matters, freshness matters, and privacy often constrains the route before quality optimization begins.

The strongest form of the thesis is:

Context engineering for coding agents is context allocation under attention, authority, freshness, privacy, trust, and cost constraints. Existing tools each solve one region of the design space. A router is the abstraction that makes those partial solutions composable.

An open, repo-resident routing layer is a promising default because it is inspectable, overrideable, local-first, and portable across agent vendors. Demonstrating its empirical advantage over fixed strategies remains the next evaluation task.

Citation

If you reference this work in academic or industry writing:

Brunner, G. *Context Routing for AI Coding Agents: A Research Agenda for Context Allocation Under Attention, Authority, Freshness, Privacy, and Cost Constraints*. ingraft project, May 2026. <https://ingraft.dev/whitepaper>

Issues, corrections, and disagreements are welcome at github.com/gunta/ingraft/issues.

This whitepaper is released under CC BY 4.0. Quote freely with attribution.

14 Appendix A: Representative Tool Snapshot, May 2026

This appendix preserves the tool-catalog value without letting the catalog drive the main argument. It is a representative snapshot, not a complete inventory.

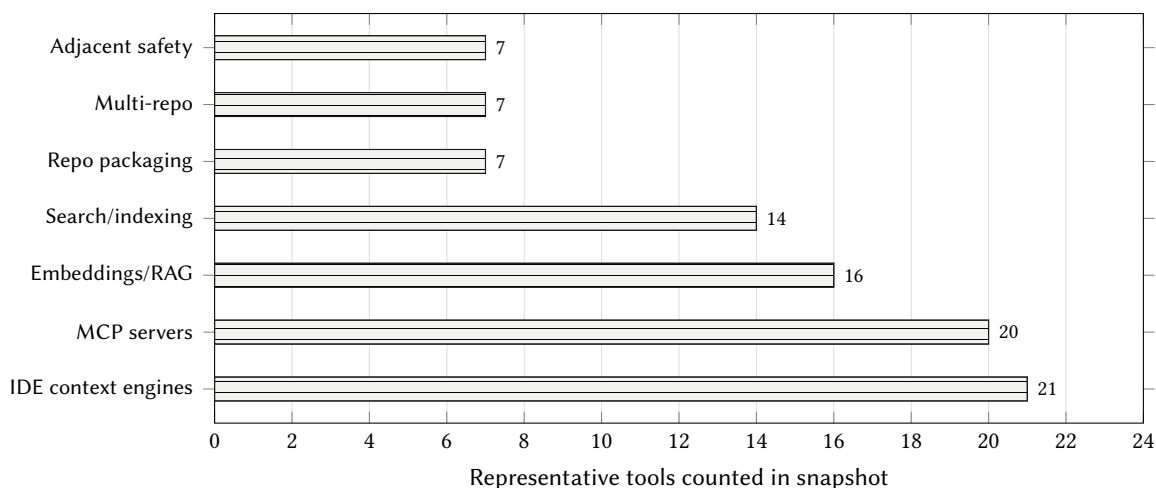


Figure 10: Tool-landscape fragmentation in the May 2026 snapshot. The distribution itself motivates routing: tools cluster around different context surfaces rather than one universal context strategy.

Table 22: Representative May 2026 tool snapshot.

Category	Representative tools	Routing lesson
IDE-native context engines	Cursor, Windsurf, GitHub Copilot, Augment, JetBrains AI/Junie, Cline, Zed, Roo Code, Continue, opencode, aider, Sourcegraph Amp, Amazon Q, Devin, OpenHands, Factory Droid, Kiro, Google Jules, Warp Agent Mode, Codebuff, Plandex	IDEs mix index, manual mention, agentic search, and rules; the router should expose which layer is active
Standalone code search/indexing	Sourcebot, Zoekt, Livegrep, Hound, ast-grep, Comby, Bloop, Greptile, Glean Code Search, Unblocked, Tabby, Serena MCP, CocoIndex, Indexify	Search wins when scale or cross-file structure overwhelms live grep
Repo packaging/snapshots	Repomix, gitingest, code2prompt, files-to-prompt, yek, ai-digest, gpt-repository-loader	Packs are useful one-shot snapshots but weak on freshness and position control
MCP context servers	Context7, filesystem MCP, GitHub MCP, GitLab MCP, DeepWiki MCP, fff, mgrep, Exa MCP, Tavily MCP, Perplexity MCP, MCPfinder, Glama, Smithery, mcp.so, Memory Bank, Mem0, Letta, Mnemos, Pieces LTM-2, OpenSrc	MCP turns context into tools; tool permissions and provenance become router concerns

Category	Representative tools	Routing lesson
Embeddings/RAG components	voyage-code-3, Jina code embeddings, Jina v2 base code, mxbai-embed-large, CodeBERT, GraphCodeBERT, UniXcoder, CodeT5+, StarCoder, CodeSage, ChromaDB, Qdrant, Pinecone, Weaviate, Milvus, pgvector	Embeddings are substrates, not policies; quality depends on selector, representation, and task
Multi-repo/org context	RepoSwarm, Backstage TechDocs, Roadie RAG, Glean, Sourcegraph, DeepWiki, Unblocked	Org context requires ACLs, provenance, and privacy-first routing
Adjacent safety/workflow	Qodo Merge, GitGuardian, ggshield AI Hook, MCP-omnisearch, GPTScript, Jujutsu, agentjj, jj-skill	Context routing touches review, secrets, web search, shared prompts, and VCS workflows

15 Appendix B: Paper Matrix

Table 23: Academic and technical works mapped to router implications.

Work	Task	Context type	Main finding	Router implication	Evidence strength
N. F. Liu et al. (2024)	QA	Long context	Position matters	Prioritize placement and bundle size	Peer-reviewed journal
Hsieh et al. (2024)	Synthetic long-context	Multi-hop long context	Effective length varies by model/task	Calibrate per model	Peer-reviewed conference
Rando et al. (2025)	Code QA and repair	Long repo context	Long context degrades unevenly on real code tasks	Avoid stuffing whole repos; evaluate by task	Preprint/benchmark
Hong et al. (2025)	Long-context reliability	Distractors, semantic needles, LongMemEval, repeated words	Increasing input length alone can make outputs less reliable	Treat context rot as a first-class routing cost	Technical report
Modarressi et al. (2025)	Long-context retrieval	Non-literal needles	Literal matching overstates long-context capability	Prefer semantic retrieval tests, not only lexical NIAH	Conference/preprint

Work	Task	Context type	Main finding	Router implication	Evidence strength
Fu et al. (2025)	Missing-information detection	Omitted context	Models struggle to identify absences even when retrieval looks easy	Do not equate retrieval with comprehension	Conference/preprint
Z. Z. Wang et al. (2025)	Code generation	Retrieved docs/code	Retrieval helps unevenly and retrievers miss useful contexts	Adaptive retrieval, not fixed top-k	Peer-reviewed findings
Thakur et al. (2021)	Information retrieval	Heterogeneous retrieval tasks	No retriever dominates zero-shot across domains	Route by task distribution and retriever failure mode	Peer-reviewed benchmark
Khattab and Zaharia (2020)	Passage search	Late interaction	Token-level matching improves retrieval while allowing pre-computation	Use late interaction when dense vectors are too lossy	Peer-reviewed conference
Formal et al. (2021)	Information retrieval	Learned sparse retrieval	Sparse expansion bridges lexical auditability and semantic matching	Include sparse search in hybrid routing	Peer-reviewed conference
Husain et al. (2019)	Code search	Natural-language to code	Code search has a language gap distinct from generic text search	Evaluate code retrieval separately from text retrieval	Benchmark/preprint
Li et al. (2024)	Code information retrieval	Multi-task code IR	Code retrieval decomposes into different task families	Do not tune one retriever for all code questions	Preprint/benchmark
Jiang et al. (2025)	Issue localization	Repository graph search	Graph-guided search can outperform flatter localization routes	Add graph search as an agentic fallback	Preprint
“U-NIAH” (2025)	Long-context/RAG	Synthetic needles/distractors	RAG helps some models and hurts others	Model-aware routing	Journal/preprint

Work	Task	Context type	Main finding	Router implication	Evidence strength
F. Zhang et al. (2023)	Code completion	Iterative repo retrieval	Repository context improves completion	Expand context iteratively	Peer-reviewed conference
Ding et al. (2023)	Code completion	Cross-file context	Single-file context is insufficient	Use repo-aware retrieval	Peer-reviewed benchmark
Liang et al. (2024)	Code completion	Fused dual context	Context quality and latency trade off	Rank, truncate, and score context	Preprint
Phan et al. (2025)	Code completion	Semantic graph	Graph retrieval outperforms flat baselines	Prefer structural retrieval when possible	Workshop/preprint
Y. (Sherry). Zhang et al. (2025)	Code RAG	AST chunks	AST chunking improves over line chunks	Structural chunking	Preprint
Brown (2026)	Practitioner source-vendoring report	Effect source tree	Locally vendored source gives agents implementation structure, tests, and examples that docs and web fetches omit	Treat source vendoring as a candidate route for high-idiom, high-centrality packages	Practitioner/vendor evidence
Gao (2026a)	Framework-agent evals	AGENTS.md docs index versus skills	Always-on compressed docs index outperformed on-demand skill use in their Next.js suite	Evaluate selector/timing, not only content quality	Vendor engineering eval
Shi et al. (2023)	Reasoning	Irrelevant context	Distractors reduce performance	Penalize distractor risk	Peer-reviewed conference

References

Agent Skills Working Group. 2025. “Agent Skills Specification.” <https://agentskills.io/specification>.

- Anthropic. 2025a. “Effective Context Engineering for AI Agents.” September 29. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. 2025b. “Equipping Agents for the Real World with Agent Skills.” October 16. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- Asai, Akari, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. “Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection.” *ICLR 2024*. <https://arxiv.org/abs/2310.11511>.
- Baumann, Nick. 2025. “Why Cline Doesn’t Index Your Codebase (and Why That’s a Good Thing).” May 27. <https://cline.bot/blog/why-cline-doesnt-index-your-codebase-and-why-thats-a-good-thing>.
- Brown, Maxwell. 2026. “The One Weird Git Trick That Makes Coding Agents More Effective.” May 11. <https://effect.website/blog/the-one-weird-git-trick-that-makes-coding-agents-more-effective/>.
- Brunner, Gunther. 2026. “ingraft: The Context Router for AI Coding Agents.” <https://ingraft.dev/>.
- Cursor. 2026. “Securely Indexing Large Codebases.” January 27. <https://cursor.com/blog/secure-codebase-indexing>.
- Ding, Yangruibo et al. 2023. “CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion.” *NeurIPS 2023 Datasets and Benchmarks Track*. <https://arxiv.org/abs/2310.11248>.
- Ding, Yangruibo, Zijian Wang, Wasi Uddin Ahmad, et al. 2024. “CoCoMIC: Code Completion by Jointly Modeling in-File and Cross-File Context.” *Proceedings of LREC-COLING 2024*. <https://arxiv.org/abs/2212.10007>.
- Gao, Jude. 2026a. “AGENTS.md Outperforms Skills in Our Agent Evals.” January 27. <https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals>.
- Gao, Jude. 2026b. “Feat(next-Codemod): Add Agents-Md Command for AI Coding Agents.” January 24. <https://github.com/vercel/next.js/pull/88961>.
- Gauthier, Paul. 2023. “Building a Better Repository Map with Tree-Sitter.” October 22. <https://aider.chat/2023/10/22/repomap.html>.
- Google, and Linux Foundation. 2025. “Agent-to-Agent Protocol (A2A).” <https://a2a-protocol.org/latest/>.
- Howard, Jeremy. 2024. “The llms.txt Specification.” September 3. <https://llmstxt.org/>.
- Hsieh, Cheng-Ping et al. 2024. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” *COLM 2024*. <https://arxiv.org/abs/2404.06654>.
- HumanLayer (Kyle). 2025. “Writing a Good CLAUDE.md.” November 25. <https://www.humanlayer.dev/blog/writing-a-good-claude-md>.
- Liang, Ming, Xiaoheng Xie, Gehao Zhang, Xunjin Zheng, Peng Di, et al. 2024. *REPOFUSE: Repository-Level Code Completion with Fused Dual Context*. <https://arxiv.org/abs/2402.14323>.

- Linux Foundation. 2025. “Linux Foundation Announces the Formation of the Agentic AI Foundation.” December. <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>.
- Liu, Nelson F., Kevin Lin, John Hewitt, et al. 2024. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for Computational Linguistics*. <https://arxiv.org/abs/2307.03172>.
- Liu, Wei et al. 2024. “GraphCoder: Code Context Graph-Based Retrieval for Repository-Level Completion.” *ASE 2024*. <https://arxiv.org/abs/2406.07003>.
- Phan, Huy N. et al. 2025. “RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion.” *FORGE 2025*. <https://arxiv.org/abs/2403.06095>.
- Rando, Stefano, Luca Romani, Alessio Sampieri, et al. 2025. *LongCodeBench: Evaluating Coding LLMs at 1M Context Windows*. arXiv preprint. <https://arxiv.org/abs/2505.07897>.
- Shi, Freda, Xinyun Chen, Kanishka Misra, et al. 2023. “Large Language Models Can Be Easily Distracted by Irrelevant Context.” *ICML 2023*. <https://arxiv.org/abs/2302.00093>.
- Shrivastava, Disha, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. *RepoFusion: Training Code Models to Understand Your Repository*. arXiv preprint. <https://arxiv.org/abs/2306.10998>.
- Sourcegraph. 2024. “SCIP — SourceCode Indexing Protocol.” <https://github.com/sourcegraph/scip>.
- Sourcegraph. 2025. “Why Sourcegraph and Amp Are Becoming Independent Companies.” December. <https://sourcegraph.com/blog/why-sourcegraph-and-amp-are-becoming-independent-companies>.
- “U-NIAH: Unified RAG and LLM Evaluation for Long Context Needle-in-a-Haystack.” 2025. *ACM Transactions on Information Systems*. <https://arxiv.org/abs/2503.00353>.
- Vercel. 2026. “next-evals-oss: Agent Evaluations for Next.js Coding Tasks.” <https://github.com/vercel/next-evals-oss>.
- Wang, Shawn (swyx), and Alessio Trotta. 2025. “Latent Space Podcast: Claude Code with Boris Cherny and Catherine Wu.” May 7. <https://www.latent.space/p/claude-code>.
- Wang, Zora Zhiruo et al. 2025. “CodeRAG-Bench: Can Retrieval Augment Code Generation?” *NAACL Findings 2025*. <https://arxiv.org/abs/2406.14497>.
- Zed Industries. 2025. “Agent Client Protocol (ACP).” <https://zed.dev/acp>.
- Zhang, Fengji, Bei Chen, Yue Zhang, et al. 2023. “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation.” *Proceedings of EMNLP 2023*. <https://arxiv.org/abs/2303.12570>.
- Zhang, Yilin (Sherry) et al. 2025. *cAST: Enhancing Code RAG with Structural Chunking via Abstract Syntax Tree*. <https://arxiv.org/abs/2506.15655>.

- Chen, Jianlv, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv preprint. <https://arxiv.org/abs/2402.03216>.
- Formal, Thibault, Benjamin Piwowarski, and Stéphane Clinchant. 2021. “SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking.” *SIGIR 2021*. <https://arxiv.org/abs/2107.05720>.
- Fu, Harvey Yiyun, Aryan Shrivastava, Jared Moore, Peter West, Chenhao Tan, and Ari Holtzman. 2025. “AbsenceBench: Language Models Can’t Tell What’s Missing.” *NeurIPS 2025 Datasets and Benchmarks Track*. <https://arxiv.org/abs/2506.11440>.
- Hong, Kelly, Anton Troynikov, and Jeff Huber. 2025. “Context Rot: How Increasing Input Tokens Impacts LLM Performance.” Chroma Technical Report. <https://trychroma.com/research/context-rot>.
- Husain, Hamel, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. *Code-SearchNet Challenge: Evaluating the State of Semantic Code Search*. arXiv preprint. <https://arxiv.org/abs/1909.09436>.
- Jiang, Zhonghao, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. *CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching*. arXiv preprint. <https://arxiv.org/abs/2503.22424>.
- Khattab, Omar, and Matei Zaharia. 2020. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.” *Proceedings of SIGIR 2020*, 39–48. <https://doi.org/10.1145/3397271.3401075>.
- Lee, Sean, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024a. “Open Source Strikes Bread – New Fluffy Embedding Model.” Mixedbread. <https://www.mixedbread.com/blog/mxbai-embed-large-v1>.
- Lee, Sean, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024b. “ColBERTus Maximus – Introducing mxbai-colbert-large-v1.” Mixedbread. <https://www.mixedbread.com/blog/mxbai-colbert-large-v1>.
- Li, Xiangyang, Kuicai Dong, Yi Quan Lee, Wei Xia, Yichun Yin, Hao Zhang, Yong Liu, Yasheng Wang, and Ruiming Tang. 2024. *CoIR: A Comprehensive Benchmark for Code Information Retrieval Models*. arXiv preprint. <https://arxiv.org/abs/2407.02883>.
- Mixedbread AI. 2026. “mgrep: Semantic Search for Agents.” <https://www.mgrep.dev/>.
- Modarressi, Ali, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. “NoLiMa: Long-Context Evaluation Beyond Literal Matching.” *International Conference on Machine Learning*. <https://arxiv.org/abs/2502.05167>.
- Mohsenimofidi, Seyedmoein, Matthias Galster, Christoph Treude, and Sebastian Baltes. 2025. *Context Engineering for AI Agents in Open-Source Software*. arXiv preprint. <https://arxiv.org/abs/2510.21413>.
- Muennighoff, Niklas, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. “MTEB: Massive Text Embedding Benchmark.” *Proceedings of EACL 2023*. <https://arxiv.org/abs/2210.07316>.

- Robertson, Stephen, and Hugo Zaragoza. 2009. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* 3 (4): 333–389. <https://doi.org/10.1561/15000000019>.
- Santhanam, Keshav, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022a. “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction.” *NAACL 2022*. <https://aclanthology.org/2022.naacl-main.272/>.
- Santhanam, Keshav, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022b. *PLAID: An Efficient Engine for Late Interaction Retrieval*. arXiv preprint. <https://arxiv.org/abs/2205.09707>.
- Thakur, Nandan, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. “BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.” *NeurIPS 2021 Datasets and Benchmarks Track*. <https://openreview.net/forum?id=wCu6T5xFjeJ>.
- Thakur, Nandan, Jimmy Lin, Sam Havens, Michael Carbin, Omar Khattab, and Andrew Drozdov. 2025. “FreshStack: Building Realistic Benchmarks for Evaluating Retrieval on Technical Documents.” *NeurIPS 2025 Datasets and Benchmarks Track*. <https://arxiv.org/abs/2504.13128>.